Operating System Principles

Module – 3

Deadlock

Deadlock

- It refers to a situation where two or more processes are unable to proceed because each is waiting for a resource that is held by another process in the same set of processes.

Deadlock – System Model

- It is a formal representation of the resources and processes involved in a deadlock situation.

- It helps in understanding the conditions and interactions that can lead to a deadlock.

Deadlock – System Model - Elements

Resources

- Entities that processes require to perform their tasks.

Resources can be classified into two types:

- **Preemptable Resources**: These resources can be taken away from a process without causing any adverse effects. Examples include memory, CPU cycles, and I/O devices.
- **Non-preemptable Resources**: These resources cannot be taken away from a process once they have been allocated. Examples include printers, CD drives, and specialized hardware.

Deadlock – System Model - Elements

Processes

- These are the executing units in an operating system. Processes can request and release resources during their execution.
- Each process has a set of activities or tasks that need to be completed.
- Process or Thread may utilize a resource in only the following sequence:
  - **Request.** The thread requests the resource. If the request cannot be granted immediately (for example, if a mutex lock is currently held by another thread), then the requesting thread must wait until it can acquire the resource.
  - **Use.** The thread can operate on the resource (for example, if the resource is a mutex lock, the thread can access its critical section).
  - **Release**. The thread releases the resource.

Deadlock – Conditions

The deadlock system model includes the four necessary conditions for deadlock to occur:

- **Mutual Exclusion**: Resources can only be used by one process at a time.
- **Hold and Wait**: Processes hold allocated resources while waiting for additional resources.
- **No Preemption**: Resources cannot be forcibly taken away from a process.
- **Circular Wait**: There exists a circular chain of processes, with each process holding a resource that is requested by the next process in the chain.

Deadlock – Resource Allocation Graph

Mutual Exclusion

At least one resource can only be used by a single process at any given time. This means that once a process acquires a resource, other processes are denied access to it until the resource is released.

Hold and Wait:

Processes hold resources while waiting for additional resources. This means that a process may be holding one or more resources and is also waiting to acquire additional resources that are currently being held by other processes.

Deadlock – Resource Allocation Graph

No Preemption:

Resources cannot be forcibly taken away from a process. They can only be released voluntarily by the process holding them.

Circular Wait:

There exists a circular chain of two or more processes, where each process is waiting for a resource that is held by another process in the chain. In other words, the last process in the chain is waiting for a resource held by the first process, creating a circular dependency.

Deadlock – System Model - Elements

Resource Allocation Graph:

- Deadlocks can be described more precisely in terms of a directed graph called a system resource-allocation graph.
- The graph consists of nodes representing processes and resources and directed edges representing the allocation and request relationships.
  - **Process Nodes**: Each process is represented by a circle or node in the graph.
  - **Resource Nodes**: Each resource is represented by a rectangle or node in the graph.
  - **Directed Edges**: An edge from a resource node to a process node represents the allocation relationship, indicating that the process currently holds the resource. An edge from a process node to a resource node represents the request relationship, indicating that the process is waiting to acquire the resource .

Deadlock

- A directed edge from thread T_i to resource type R_j is denoted by $T_i \rightarrow R_j$; it signifies that thread T_i has requested an instance of resource type R_j and is currently waiting for that resource.
- A directed edge from resource type R_j to thread T_i is denoted by $R_j \rightarrow T_i$; it signifies that an instance of resource type R_j has been allocated to thread T_i. A directed edge $T_i \rightarrow R_j$ is called a **request edge**; a directed edge $R_j \rightarrow T_i$ is called an **assignment edge**.

Deadlock – Resource Allocation Graph

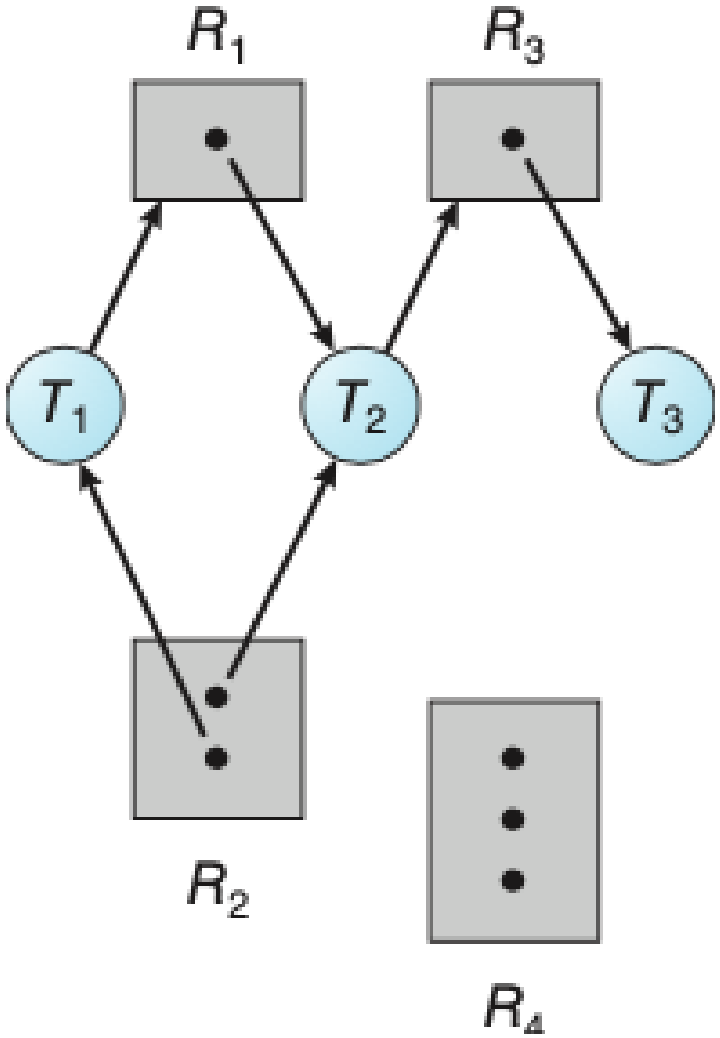

Figure Resource-allocation graph

Deadlock – Resource Allocation Graph

- When thread T_i requests an instance of resource type R_j, a request edge is inserted in the resource-allocation graph.
- When this request can be fulfilled, the request edge is ***instantaneously*** transformed to an assignment edge.

Deadlock – Resource Allocation Graph

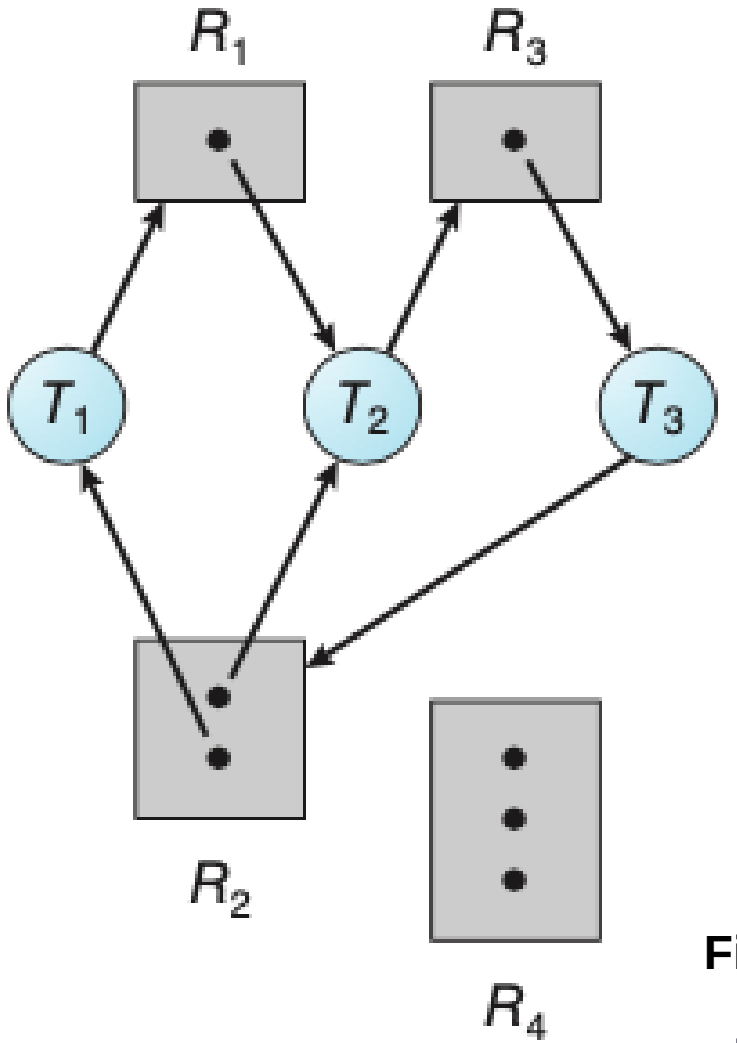

Figure Resource-allocation graph

Deadlock – Resource Allocation Graph

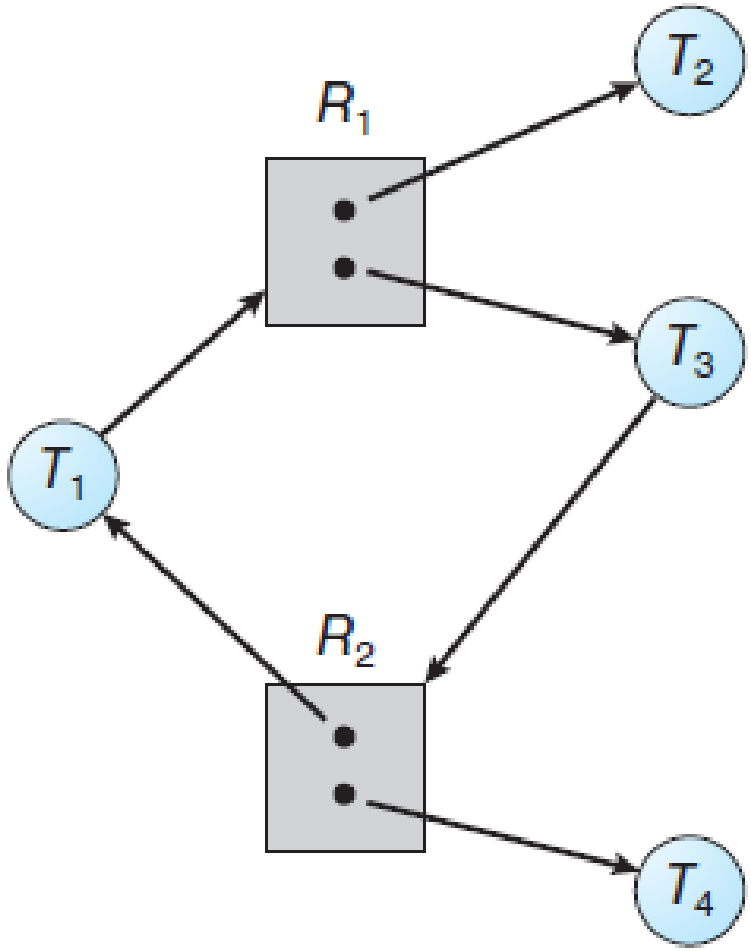

Figure: Resource-allocation graph with a cycle but no deadlock.

Deadlock – Resource Allocation Graph

- If a resource-allocation graph does not have a cycle, then the system is not in a deadlocked state.

- If there is a cycle, then the system may or may not be in a deadlocked state.

Deadlock – Methods for Handling Deadlocks

- When the four conditions are satisfied, a deadlock can occur. As a result, none of the processes involved can make progress, leading to a system deadlock.
- Deadlock problem can be dealt in one of three ways:
  - We can ignore the problem altogether and pretend that deadlocks never occur in the system.
  - We can use a protocol to prevent or avoid deadlocks, ensuring that the system will never enter a deadlocked state.
  - We can allow the system to enter a deadlocked state, detect it, and recover.

Deadlock – Methods for Handling Deadlocks

- Deadlocks can be prevented, avoided, or resolved using various techniques, such as
  - Resource allocation strategies,
  - Deadlock detection algorithms, and
  - Deadlock recovery mechanisms.
- The goal is to ensure that the system remains in a safe state where deadlocks cannot occur, or if they do occur, they can be resolved efficiently to restore system functionality.

Deadlock Prevention

- Ensure that any one condition - *Mutual Exclusion, No preemption, Hold and Wait, circular wait* does not hold. Then occurrences of the Deadlock can be prevented.
- It is a proactive approach.
- It aims to identify and eliminate the conditions that lead to deadlocks by enforcing restrictions on resource allocation and process execution.

Deadlock Prevention

Mutual Exclusion Avoidance:

- Ensure that resources are shareable whenever possible.
- If multiple processes can access a resource simultaneously without interfering with each other's operations, the issue of mutual exclusion is eliminated.
- For resources that cannot be shared, consider allowing multiple processes to have read-only access while enforcing exclusive access for write operations.

Deadlock –Prevention

Hold and Wait Avoidance:

- Implement a strategy where:
  - Processes must request and acquire all the necessary resources they need before starting execution. This can be achieved by using a resource allocation protocol known as "*all or nothing*" *or "claim all*" strategy.
  - Another protocol allows a thread to request resources only when it has none. Before it can request any additional resources, it must release all the resources

Disadvantages: resource utilization may be low, starvation is possible

Deadlock – Management - Prevention

No Preemption Avoidance:

To ensure that this condition does not hold, we can use the following protocol:

- If a thread is holding some resources and requests another resource that cannot be immediately allocated to it, then all resources the thread is currently holding are preempted.
- The thread will be restarted only when it can regain its old resources, as well as the new ones that it is requesting

Deadlock – Prevention

Circular Wait Avoidance:

- Introduce a total ordering of resources and impose a rule that each thread can only request resources in an increasing order. This eliminates the possibility of circular wait by ensuring that a thread always requests resources in a consistent order.
- At first, implement a resource hierarchy, where each resource is assigned a unique identifier, and threads can only request resources with a lower identifier. This ensures a partial ordering of resources and prevents circular wait.

Deadlock – Prevention

Circular Wait Avoidance:

- Secondly, a thread requesting an instance of resource must have released any resources
- If several instances of the same resource type are needed, a single request for all of them must be issued.